School of Computing

Assessment Brief

Outline	Basic information
Module code and title:	COMP6018 - Theory and Practice of Concurrency
Who to Contact Name & Email Address:	Laura Bocchi (l.bocchi@kent.ac.uk)
Assessment name (if applicable):	Assessment 2
Assessment type and component:	Practical Project
Assessment weighting:	40%
Module learning outcomes assessed in this assignment:	Have a critical understanding of the principles of concurrent programming, as well as its advantages and challenges. Critically evaluate and appraise the properties of a distributed process (e.g. safety and liveness), and compare the behaviour of different processes. Apply the principles of concurrency theory to real scenarios. Apply simple abstract concepts of concurrency theory to master complex concrete scenarios.
Submission deadline:	Wed, 26 Nov 2025
Where to submit:	Moodle
How to request an extension:	Unexpected events such as illness, an accident, or bereavement sometimes happens. If so, you may need to request an extension, late submission or to be absent on the day of an assessed event. For instructions, see "in-course extenuating circumstance" in My Kent.
When marks and feedback on your submission will be provided on Kent Vision:	Wed, 10 Dec 2025
How to find your feedback in Moodle	Your tutor's feedback on your assessment is one of the most valuable learning resources you have on your course. Please review it, compare it with feedback you've received on other assignments, and use it to identify skills where you can improve on future assignments. Your tutor, academic advisor, or a member of

	the Skills for Academic Success (SAS) team can help you apply the lessons in this feedback to future work.

1. The Task

1.1. What do I need to do?

In this assessment you will work on a problem, using synchronous and asynchronous message passing in Go. You will also be asked to reflect on the properties of your solutions. The assessment consists of **three parts**.

Part 1 (50%)

You need to implement a system for drop in sessions run by one single lawyer. The lawyer is sitting in her office, waiting to meet clients for one-to-one meetings. Outside the office there is a small waiting room with space for ***n*** clients to stand in a queue.

- The **lawyer** checks if any client is in the waiting room. If there are no clients she sits at her desk and reads some papers (i.e., sleeps). If there are clients in the waiting room, she calls one in. The remaining clients, if any, keep waiting. When the lawyer finishes talking to a client, she dismisses them and checks if there are other clients in the waiting room. And so on …
- The **client**, upon arrival, checks if the lawyer is busy with other clients or reading. If the lawyer is reading, the client wakes her up and the meeting starts. If the lawyer is busy with another client, the arriving client goes in the waiting room and wait (i.e., sleeps).

To model the drop in session problem, you will need to create a lawyer function, and a client function. Use channels to synchronize the activities of client and lawyer. Here are the signatures of the functions:

```
func lawyer(waitRoom chan chan string, read chan chan string)
func client(waitRoom chan chan string, read chan chan string, name
string)
```

Here is a model of the client function in **pi calculus** that describes the behaviour of function client (implement the message *met* as print statement in Go). Function lawyer should be dual to client.

Client = ν x.($\overline{\texttt{waitRoom}}$ x. x. **SMeet** + $\overline{\texttt{read}}$ x. **SMeet**)

SMeet = (x . $\overline{\texttt{met}}$ .0)

Constraints/hints.

1. Use message passing (no shared memory, counters, arrays, …).
2. Channel *read* models the state reading/meeting of the lawyer: the lawyer is reading when she is blocked on *read*, otherwise she is in a meeting.
3. Channel *waitRoom* models access to the waiting room.
4. Channels *x* model the state (waiting/meeting) of a client. Each client creates a fresh *x* (type *chan string*) and passes it to the lawyer on channels *waitRoom* or *read*. When the client is blocked on *x* they are either waiting in the waiting room or listening to the lawyer in a meeting.
5. Implement the following **priorities**:
    a. The lawyer should never start reading if there is at least one client in the waiting room
    b. A client should never go in the waiting room if the lawyer is reading.
6. Allow the meeting to take some random time (using *time.Sleep()*).
7. You can build your solution using the following template https://go.dev/play/p/wmIIUYAVElY and get something like the following:

```
Diverdark met directly
Frightvivid met directly
Scourgeazure met after waiting
Painterdew met after waiting
Rayred met after waiting
Scribeolive met after waiting
…

Program exited.
```

Part 2 (35%)

The situation outside the office is going out of hand. The clients that did not find a place in the waiting room are forming a crowd in the corridor. Some started arguing, as to who should get in next. Personnel in the nearby offices urge the lawyer to find a solution. The lawyer writes a note at the entrance of the waiting room: "If the waiting room is full, please leave and take appointment by email".

Provide a go implementation (of the whole system) where clients check if the waiting room is full without blocking on *waitRoom*. If *waitRoom* is full they go away and arrange a meeting on a channel *mailBox* that is the inbox of the lawyer. See below for a pi calculus model (*SMeet* is as in Part 1)

$$
\begin{aligned}
\textbf{Client2} \;=\; & \nu\, x.\,(\overline{\texttt{waitRoom}}\ x.\ x.\ \textbf{SMeet} \\
& + \\
& \overline{\texttt{read}}\ x.\ \textbf{SMeet} \\
& + \\
& \nu\, \texttt{teams}.\ \overline{\texttt{mailBox}}\ \texttt{teams}.\ \overline{\texttt{teams}}\ x\ .\ \textbf{SMeet} \\
&)
\end{aligned}
$$

Constraints/hints.

1. All clients try to meet in person before writing an email.
2. The lawyer prioritises clients in the waiting room. She meets those waiting on the mailbox only when the waiting room is empty.
3. Some client points out that the current policy may lead to a starvation problem. In your solution to this part, **provide a main function that shows an occurrence of starvation** for some (at least one) clients.
4. As a code comment, provide a **pi calculus** model of the whole system as the parallel composition of Client2 and the corresponding lawyer process.

Part 3 (15%)

Provide a variant of your solution in part 2 where the lawyer uses ageing to prevent starvation. At regular intervals of time, the lawyer upgrades the first client in the mailbox queue (if any) into the last seat in the waiting room. Try to avoid deadlocks.

1.2. Where do I start? Tips for success.

Read the assessment brief – see **Section 1.1 What do I need to do?**

Start early: give yourself enough time to complete the assessment.

Look at the lecture slides and the class materials. on Go and on Properties. It will help revise them before attempting these tasks.

1.3. May I use Generative AI?

See the latest guidance at the My Kent page on Generative AI use in assessments.

The use of Generative AI is not permitted, as it would invalidate the learning outcomes (problem solving) and likely produce solutions that are too generic for the purpose.

1.4. What will I submit? Are there any special formatting or referencing styles expected?

Your solution needs to be submitted on Moodle as one zip file xxx.zip (where xxx is your id) containing the following files:

- xxx_part1.go (your solution to part 1)
- xxx_part2.go (your solution to part 2)
- xxx_part3.go (your solution to part 3)

Please only include files of the solutions you are attempting.

1.5. I have an Inclusive Learning Plan (ILP), how will this plan be accommodated?

In general, assessments have been designed to be inclusive. For timed assessments, extra time will automatically be given if your ILP recommends it. If you need more time for a coursework assessment due to the impact of your disability or mental health condition, please contact the student engagement team for your School to request an in-course extenuating circumstance. If you have an ILP that you think requires further accommodation, please contact the module convenor to discuss how it would work.

2. Support Available

2.1. Module activities or tasks that specifically help you prepare you for this assignment.

Lectures in the 13-15th week of term will provide the foundational tools and skills needed for this assignment. The class exercises in week 13th are designed to help you prepare for this assignment.

2.2. Further assistance

For questions related to this assignment, you can contact Laura Bocchi (l.bocchi@kent.ac.uk).

See My Kent for links to support services and resources to help you with your studies. These include the Course Administration Team, Library Search and Subject Librarians, Skills for Academic Success, and Mental Health and Wellbeing Support.

3. Marking Criteria- How will my assignment be marked?

Part 1 (50%):

- Implementation of function client [15] and lawyer [10] that respects the signature and is faithful to the given model in pi calculus and textual specification.

- Correct implementation of priorities among the waiting channels for client [9] and lawyer [9].
- Main function with meaningful running test and neat prints in function client [6].

Part 2 (35%)

- Implementation of function client [8] and lawyer [7] that respects the signature and is faithful to the given model in pi calculus and textual specification.
- Correct CCS model [14] (correct syntax [5], correct logic [9])
- The main runs correctly and shows starvation [6]

Part 3 (15%)

- Correct implementation of ageing [10]
- No potential deadlocks [5]